RAPPORT DE TRAVAUX PRATIQUES

Internet des objets — Atelier 6, partie 1

# Smart Room Monitoring

Chaîne IoT complète et supervision temps réel

## Acquisition DHT22, transmission ESP32, serveur Node.js et tableau de bord web

Ce rapport présente la conception, la réalisation et la validation d'une chaîne d'Internet des objets permettant de mesurer la température et l'humidité d'une salle, de transmettre les données à un serveur et de les visualiser en temps réel dans un navigateur.

**DHT22** → **ESP32** → **Node.js / Express** → **Socket.IO** → **Dashboard**

PRÉSENTÉ PAR

**Youssef El Ajbari et Monssef Bougaidan**

3e année — IDAI

Module : Internet des objets

30 octobre 2026

Année universitaire 2026–2027

ESP32 · DHT22 · HTTP/JSON · WebSocket · Node.js

# Table des matières

1 Introduction 1

2 Architecture générale 1

3 Firmware ESP32 2

4 Serveur Node.js 5

5 Tableau de bord web 8

6 Tests et résultats 11

# 1 Introduction

## 1.1 Contexte

Les ateliers précédents ont introduit la commande d'une LED, la lecture d'un capteur DHT22 et une première logique locale de « Smart Room ». L'atelier 6 change d'échelle : il ne s'agit plus seulement de faire clignoter une sortie ou d'afficher une mesure sur un écran local, mais de construire une chaîne complète, depuis le capteur jusqu'à une interface consultable à distance.

Le système réalisé, nommé **Smart Room Monitoring**, poursuit un objectif simple : rendre disponible en temps réel, dans un navigateur, la température et l'humidité d'une salle. Cette contrainte impose de résoudre plusieurs problèmes complémentaires : acquisition fiable, transmission réseau, validation des données, stockage temporaire, diffusion événementielle et présentation ergonomique.

**Objectifs de l'atelier**

- acquérir la température et l'humidité avec un capteur DHT22 ;
- transmettre les mesures en Wi-Fi vers un serveur avec un protocole HTTP et un format JSON ;
- développer un serveur Node.js qui valide, mémorise et rediffuse les mesures en temps réel ;
- concevoir un tableau de bord web lisible, réactif et doté d'alertes ;
- valider la chaîne complète et analyser les résultats expérimentaux.

## 1.2 Cahier des charges

Le tableau 1 résume les exigences retenues et les critères d'acceptation associés. Ces critères ont guidé les choix techniques et servi de base aux essais de la partie 6.

| Exigence | Critère d'acceptation |
|---|---|
| **Acquisition** | Température et humidité lues automatiquement toutes les 5 s. |
| **Transmission** | Envoi HTTP POST d'un objet JSON vers `/api/measurements`, avec réponse 200 en cas de succès. |
| **Validation** | Rejet par un code 400 des données manquantes, mal typées ou hors plage. |
| **Temps réel** | Mise à jour du tableau de bord sans rechargement, via Socket.IO. |
| **Historique** | Conservation et affichage des dix dernières mesures. |
| **Alertes** | Signalement visuel si la température dépasse 30 °C ou si l'humidité dépasse 70 %. |
| **Interface** | Page responsive, état de connexion visible, horodatage des mesures. |

TABLE 1 – Cahier des charges fonctionnel de l'atelier.

# 2 Architecture générale

## 2.1 Vue d'ensemble

L'architecture suit un modèle en trois couches, représenté sur la figure 1 :

- **couche de perception** : le capteur DHT22 mesure la température et l'humidité, l'ESP32 les acquiert et les met en forme ;
- **couche de traitement** : le serveur Node.js reçoit les requêtes HTTP, valide les données, les

stocke en mémoire et les rediffuse ;

— **couche de présentation** : le navigateur affiche le tableau de bord et reçoit les mises à jour par WebSocket.




**Figure 1 –** Architecture en trois couches de la chaîne Smart Room Monitoring.

Le serveur joue le rôle de hub. Il expose une API REST pour l'écriture et la lecture des mesures, et un canal WebSocket pour la diffusion. Ce découpage presente deux avantages : le microcontrôleur n'a pas besoin de connaître les navigateurs connectés, et plusieurs clients peuvent observer simultanément les mêmes données.

# 3 Firmware ESP32

## 3.1 Câblage et configuration

Le DHT22 est relié à l'ESP32 selon le tableau 2. Le choix du GPIO 17 est arbitraire mais documenté dans le code ; il faut simplement éviter les broches réservées au démarrage ou au port série.

| DHT22 | ESP32 | Remarque |
|---|---|---|
| VCC | 3V3 | Alimentation en 3,3 V. |
| DATA | GPIO 17 | Ligne de données. |
| GND | GND | Masse commune. |

**Table 2 –** Connexions entre le capteur DHT22 et l'ESP32.

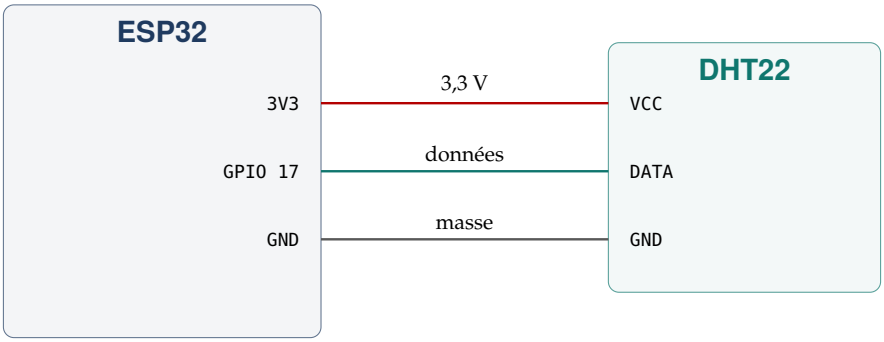



**Figure 2 –** Schéma de principe du câblage DHT22 – ESP32.

## 3.2 Bibliothèques et paramètres

Le firmware s'appuie sur quatre bibliothèques : `WiFi.h` pour la connexion réseau, `HTTPClient.h` pour les requêtes HTTP, `DHT.h` pour le capteur et `ArduinoJson.h` pour la sérialisation. Les para-

mètres globaux sont regroupés au début du fichier (code 1).

```
#include <WiFi.h>
#include <HTTPClient.h>
#include "DHT.h"
#include <ArduinoJson.h>

#define DHTPIN 17
#define DHTTYPE DHT22

DHT dht(DHTPIN, DHTTYPE);

const char* ssid = "Wifi";
const char* password = "psswd";

const char* serverURL = "http://10.198.217.139:3000/api/measurements";

const char* deviceId = "ESP32_01";
const char* location = "Salle TP1";
const char* sensorType = "DHT22";

unsigned long previousMillis = 0;
const unsigned long interval = 5000;
unsigned long counter = 0;
```

**Code 1 –** Configuration et paramètres du firmware (`esp32/main.c`).

Les identifiants Wi-Fi et l'adresse du serveur sont volontairement génériques dans cet extrait. Dans un déploiement réel, ils doivent être placés dans un fichier de configuration non versionné ou saisis via un portail de configuration. L'adresse `10.198.217.139` correspond à l'IP de la machine qui exécute le serveur sur le réseau local : `localhost` ne peut pas être utilisé depuis l'ESP32, car il désignerait le microcontrôleur lui-même.

## 3.3 Acquisition et envoi

La fonction `sendMeasurements` (code 2) construit l'objet JSON, l'envoie au serveur et journalise le résultat sur la liaison série. Le compteur `counter` est incrémenté à chaque appel ; il permet de vérifier qu'aucune mesure n'a été perdue entre deux envois.

```
void sendMeasurements(float temperature, float humidity) {
  JsonDocument doc;

  counter++;

  doc["device_id"] = deviceId;
  doc["temperature"] = temperature;
  doc["humidity"] = humidity;
  doc["location"] = location;
  doc["sensor_type"] = sensorType;
  doc["counter"] = counter;
  doc["status"] = "ACTIVE";

  String jsonData;
  serializeJson(doc, jsonData);

  Serial.println(jsonData);

  if (WiFi.status() != WL_CONNECTED) {
    Serial.println("Wi-Fi deconnecte, envoi annule");
```

```
    return;
  }

  HTTPClient http;

  http.begin(serverURL);
  http.addHeader("Content-Type", "application/json");

  int code = http.POST(jsonData);

  Serial.print("Code HTTP : ");
  Serial.println(code);

  if (code > 0) {
    Serial.println(http.getString());
  }

  http.end();
}
```

**Code 2 –** Construction et envoi d'une mesure (`esp32/main.c`).

Le corps de la requête est sérialisé dans une `String` avant l'envoi. Le type `Content-Type: application/json` est indispensable pour qu'Express analyse automatiquement le corps avec `express.json()`. Le code HTTP est affiché sur la liaison série : une valeur strictement positive indique qu'une réponse a été reçue, tandis qu'une valeur négative signale une erreur de connexion.

## 3.4 Boucle d'acquisition

La boucle principale (code 3) utilise `millis()` plutôt qu'un `delay(5000)`. Cette approche non bloquante laisse la boucle disponible pour d'autres traitements et rend le rythme d'acquisition indépendant du temps d'envoi.

```
void setup() {
  Serial.begin(115200);
  dht.begin();

  WiFi.begin(ssid, password);

  Serial.println("Connexion Wi-Fi...");

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println();
  Serial.println("Wi-Fi connecte !");
  Serial.println(WiFi.localIP());
}

void loop() {
  if (millis() - previousMillis >= interval) {
    previousMillis = millis();

    float temperature = dht.readTemperature();
    float humidity = dht.readHumidity();

```

```
    if (isnan(temperature) || isnan(humidity)) {
      Serial.println("Erreur DHT22");
    } else {
      sendMeasurements(temperature, humidity);
    }
  }
}
```

**Code 3 –** Initialisation et boucle principale (`esp32/main.c`).

### 3.5 Gestion des erreurs

Trois cas d'erreur sont traités explicitement :

— **lecture capteur invalide** : si `isnan()` détecte une valeur non numérique, la mesure est ignorée et un message « Erreur DHT22 » est écrit sur la liaison série ;
— **Wi-Fi indisponible** : si la connexion est perdue, l'envoi est annulé et un message le signale ; aucune donnée n'est mise en file d'attente ;
— **échec HTTP** : le code retourné par `http.POST()` est journalisé. Un code négatif signifie que la requête n'a pas abouti ; un code 400 ou 500 signifie que le serveur a répondu mais a refusé ou échoué.

Le firmware ne réessaie pas automatiquement un envoi échoué. Cette limitation est acceptable pour un prototype, mais elle explique les codes `-1` observés en fin de première série d'essais (figure 4). Une stratégie de reconnexion avec backoff est proposée dans les perspectives.

## 4 Serveur Node.js

### 4.1 Initialisation

Le serveur repose sur Express pour le routage HTTP et sur Socket.IO pour la diffusion temps réel. Il sert également les fichiers statiques du tableau de bord, ce qui évite de gérer deux origines et simplifie le déploiement (code 4).

```
const express = require("express");
const http = require("http");
const { Server } = require("socket.io");

const app = express();
const server = http.createServer(app);
const io = new Server(server);

app.use(express.json());
app.use(express.static("public", { index: "dashboard.html" }));

const HISTORY_SIZE = 10;

let lastMeasurement = null;
let history = [];
```

**Code 4 –** Initialisation du serveur Express et de Socket.IO (`nodeserver/server.js`).

Le serveur écoute sur le port 3000 et sur l'interface `0.0.0.0`, afin d'être joignable depuis les autres machines du réseau local, notamment l'ESP32. Le tableau de bord est servi à la racine `/` grâce à l'option `index: "dashboard.html"`.

## 4.2 API REST

L'API expose trois routes principales, résumées dans le tableau 3.

| Méthode | Route | Rôle | Réponse |
|---|---|---|---|
| `POST` | `/api/measurements` | Réception et validation d'une mesure | 200 ou 400 |
| `GET` | `/api/measurements/latest` | Dernière mesure connue | Objet ou `null` |
| `GET` | `/api/measurements/history` | Historique des dix dernières mesures | Tableau JSON |
| `GET` | `/` | Tableau de bord statique | HTML |

**Table 3 –** Routes exposées par le serveur.

## 4.3 Validation des données

Express analyse le corps JSON, mais ne vérifie ni les types ni les plages de valeurs. Une fonction dédiée, `validateMeasurement`, centralise donc les contrôles (tableau 4). Elle retourne un tableau d'erreurs ; si ce tableau n'est pas vide, le serveur répond avec un code 400 et la liste des motifs.

| Champ | Contrainte | Message d'erreur associé |
|---|---|---|
| `device_id` | chaîne non vide, obligatoire | « device_id est obligatoire et doit etre une chaine de caracteres » |
| `temperature` | nombre fini, entre -40 et 80 | « temperature doit etre un nombre » ou « temperature doit etre comprise entre -40 et 80 degres » |
| `humidity` | nombre fini, entre 0 et 100 | « humidity doit etre un nombre » ou « humidity doit etre comprise entre 0 et 100 % » |
| `location` | chaîne, facultative | « location doit etre une chaine de caracteres » |
| `sensor_type` | chaîne, facultative | « sensor_type doit etre une chaine de caracteres » |
| `counter` | entier positif ou nul, facultatif | « counter doit etre un entier positif ou nul » |
| `status` | chaîne, facultative | « status doit etre une chaine de caracteres » |

**Table 4 –** Règles de validation appliquées par le serveur.

## 4.4 Traitement d'une mesure

Le gestionnaire de la route POST (code 5) applique la validation, normalise les chaînes, complète les valeurs absentes et construit un objet `measurement`. Il met ensuite à jour la dernière mesure et l'historique, journalise l'événement dans la console, diffuse la mesure aux clients et renvoie la réponse HTTP.

```
62 app.post("/api/measurements", (req, res) => {
63   const errors = validateMeasurement(req.body);
64
65   if (errors.length > 0) {
66     return res.status(400).json({
67       success: false,
68       message: "Donnees invalides",
69       errors
70     });
71   }
72
73   const data = req.body;
74
75   const measurement = {
```

```
    device_id: data.device_id.trim(),
    temperature: data.temperature,
    humidity: data.humidity,
    location:
      typeof data.location === "string" && data.location.trim() !== ""
        ? data.location.trim()
        : "Salle inconnue",
    sensor_type:
      typeof data.sensor_type === "string" && data.sensor_type.trim() !== ""
        ? data.sensor_type.trim()
        : "DHT22",
    counter: data.counter !== undefined ? data.counter : null,
    status:
      typeof data.status === "string" && data.status.trim() !== ""
        ? data.status.trim()
        : "ACTIVE",
    receivedAt: new Date().toISOString()
  };

  lastMeasurement = measurement;

  history.unshift(measurement);
  if (history.length > HISTORY_SIZE) {
    history.pop();
  }

  console.log("Mesure recue :", measurement);

  io.emit("newMeasurement", measurement);
  io.emit("history", history);

  res.status(200).json({
    success: true,
    message: "Mesure recue",
    counter: measurement.counter
  });
});
```

**Code 5 –** Gestion d'une requête POST et diffusion temps réel (`nodeserver/server.js`).

Deux détails sont importants pour la cohérence du système. D'une part, les valeurs par défaut expliquent pourquoi un payload minimal à trois champs produit tout de même une mesure complète : `location` devient « Salle inconnue », `sensor_type` devient `DHT22`, `status` devient `ACTIVE` et `counter` reste `null`. C'est exactement ce que montrent les figures 5 et 3. D'autre part, l'historique est borné à dix éléments : `unshift` ajoute en tête et `pop` supprime l'élément le plus ancien.

## 4.5 Stockage et diffusion

Le stockage est entièrement en mémoire. Les variables `lastMeasurement` et `history` sont initialisées à `null` et `[]` au démarrage ; elles sont perdues à l'arrêt du processus. Ce choix simplifie le prototype et suffit pour les essais, mais il ne permet pas de conserver un historique après redémarrage.

La diffusion repose sur deux événements Socket.IO :

— `newMeasurement` : envoyé à tous les clients à chaque nouvelle mesure ;

— `history` : envoyé après chaque mesure et à la connexion d'un nouveau client, pour lui transmettre l'état courant.

Lorsqu'un navigateur se connecte, le serveur lui envoie immédiatement la dernière mesure et l'historique (code 6). Le tableau de bord peut donc afficher des données dès l'ouverture, sans attendre la mesure suivante.

```
app.get("/api/measurements/latest", (req, res) => {
  res.json(lastMeasurement);
});

app.get("/api/measurements/history", (req, res) => {
  res.json(history);
});

io.on("connection", (socket) => {
  console.log("Dashboard connecte :", socket.id);

  if (lastMeasurement) {
    socket.emit("newMeasurement", lastMeasurement);
  }

  socket.emit("history", history);

  socket.on("disconnect", () => {
    console.log("Dashboard deconnecte :", socket.id);
  });
});

server.listen(3000, "0.0.0.0", () => {
  console.log("Serveur actif sur http://0.0.0.0:3000");
});
```

**Code 6 –** Lecture de l'état courant et gestion des connexions Socket.IO (`nodeserver/server.js`).

# 5 Tableau de bord web

## 5.1 Interface

Le tableau de bord est une page HTML unique, servie par Express. La figure 3 montre l'interface en fonctionnement. Elle est organisée en quatre zones :

— un en-tête avec le titre, le sous-titre et un badge de connexion (« Serveur connecté » ou « Serveur déconnecté ») ;
— une série de cartes d'information : salle, équipement, capteur, état, compteur et dernière réception ;
— une zone d'alertes, verte lorsque les valeurs sont dans les seuils, rouge sinon ;
— deux cartes de mesure (température et humidité) et un tableau d'historique.

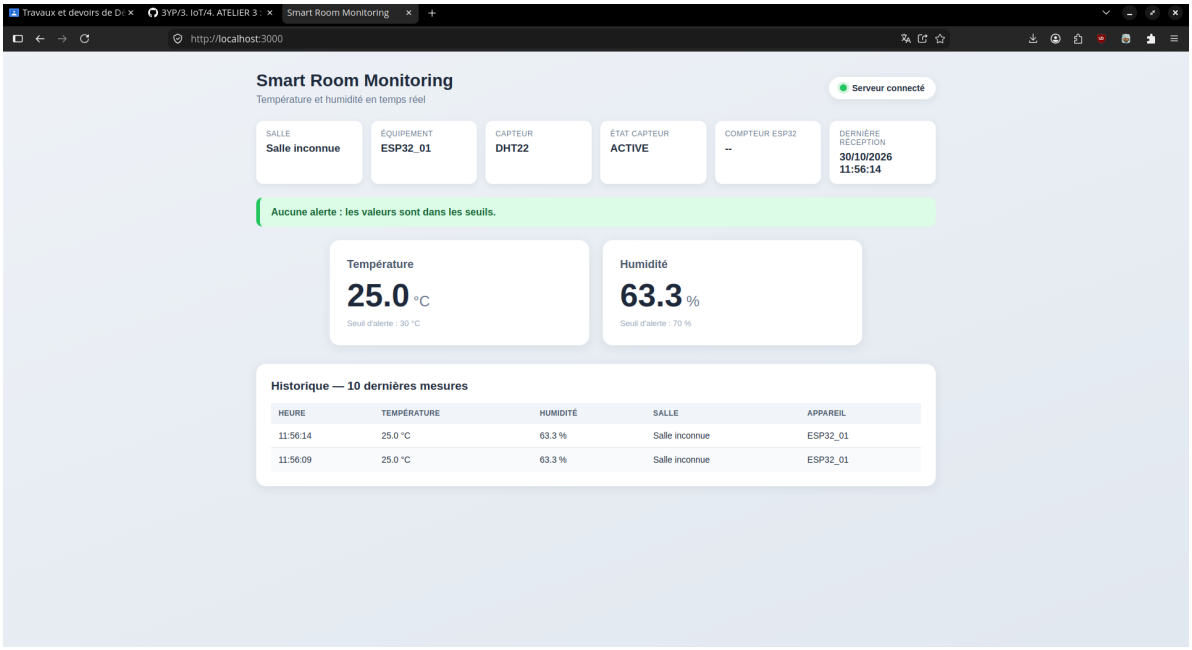



**FIGURE 3 –** Tableau de bord en fonctionnement : température 25,0 °C, humidité 63,3 %, serveur connecté.

La capture montre une mesure à 25,0 °C et 63,3 % d'humidité, avec la salle « Salle inconnue », l'équipement `ESP32_01`, le capteur `DHT22`, l'état `ACTIVE` et un compteur à `-`. Ces valeurs illustrent le mécanisme de valeurs par défaut décrit dans la partie 4 : le payload envoyé pendant l'essai ne contenait que `device_id`, `temperature` et `humidity`. La dernière réception est horodatée au 30/10/2026 à 11 :56 :14, ce qui correspond à 10 :56 :14 UTC dans le journal serveur (figure 5), le navigateur affichant l'heure locale (UTC+1).

## 5.2 Logique cliente

Le script client ouvre une connexion Socket.IO avec `io()`, écoute les événements `newMeasurement` et `history`, puis met à jour le DOM. La fonction `displayMeasurement` (code 7) renseigne les cartes et déclenche les alertes. Le tableau de bord utilise `toFixed(1)` pour afficher une décimale et `toLocaleTimeString("fr-FR", {hour12:false})` pour un horodatage en 24 h.

```
function applyAlerts(temperature, humidity) {
  const temperatureAlert = temperature > TEMPERATURE_THRESHOLD;
  const humidityAlert = humidity > HUMIDITY_THRESHOLD;

  document
    .getElementById("temperature-card")
    .classList.toggle("alert", temperatureAlert);

  document
    .getElementById("humidity-card")
    .classList.toggle("alert", humidityAlert);

  const alerts = [];

  if (temperatureAlert) {
    alerts.push(
      `Température trop élevée : ${temperature.toFixed(1)} °C ` +
      `(seuil : ${TEMPERATURE_THRESHOLD} °C)`
    );
  }

```

```
  if (humidityAlert) {
    alerts.push(
      'Humidité trop élevée : ${humidity.toFixed(1)} % ' +
      '(seuil : ${HUMIDITY_THRESHOLD} %)'
    );
  }

  const container = document.getElementById("alerts");

  if (alerts.length === 0) {
    container.innerHTML =
      '<div class="alert ok">Aucune alerte : les valeurs sont dans les seuils.</div>';
    return;
  }

  container.innerHTML = alerts
    .map((message) => '<div class="alert">! ${message}</div>')
    .join("");
}

function displayMeasurement(data) {
  const temperature = Number(data.temperature);
  const humidity = Number(data.humidity);

  document.getElementById("temperature").textContent = temperature.toFixed(1);
  document.getElementById("humidity").textContent = humidity.toFixed(1);

  document.getElementById("room").textContent = data.location || "--";
  document.getElementById("device").textContent = data.device_id || "--";
  document.getElementById("sensor").textContent = data.sensor_type || "--";
  document.getElementById("sensor-status").textContent = data.status || "--";
  document.getElementById("counter").textContent =
    data.counter !== undefined && data.counter !== null ? data.counter : "--";
  document.getElementById("time").textContent = formatDateTime(data.receivedAt);

  applyAlerts(temperature, humidity);
}
```

**Code 7 –** Affichage d'une mesure et calcul des alertes (`nodeserver/public/dashboard.html`).

## 5.3 Seuils et alertes

Les seuils sont définis en tête de script : 30 °C pour la température et 70 % pour l'humidité (tableau 5). Ils sont volontairement simples et adaptés à une salle de travaux pratiques ; ils pourraient être rendus configurables depuis l'interface.

| Grandeur | Seuil | Comportement de l'interface |
|---|---|---|
| Température | $> 30$ °C | Bordure rouge sur la carte et message d'alerte explicite. |
| Humidité | $> 70$ % | Bordure rouge sur la carte et message d'alerte explicite. |
| Aucune alerte | valeurs dans les seuils | Bandeau vert « Aucune alerte ». |

**Table 5 –** Seuils d'alerte du tableau de bord.

Avec 25,0 °C et 63,3 %, les deux grandeurs sont dans les seuils : la capture de la figure 3 affiche donc le bandeau vert et aucune bordure rouge.

### 5.4 Initialisation et reconnexion

Au chargement, le script exécute deux requêtes en parallèle : `/api/measurements/latest` et `/api/measurements/history`. Si l'historique est disponible, il est affiché et la mesure la plus récente est utilisée pour remplir les cartes ; sinon, la dernière mesure est affichée seule. Cette stratégie évite un écran vide après un rechargement de page.

Socket.IO gère automatiquement la reconnexion du navigateur. Les événements `connect` et `disconnect` mettent à jour le badge ; lorsque la connexion revient, le serveur renvoie l'état courant. Le client ne conserve pas de copie persistante des mesures : il reconstruit son affichage à partir du serveur.

### 5.5 Prévention des injections HTML

Les champs provenant du serveur sont échappés avant insertion dans le tableau d'historique grâce à la fonction `escapeHtml`. Cette précaution évite qu'un identifiant d'appareil ou un nom de salle contenant du HTML ne soit interprété par le navigateur. Elle est particulièrement importante si le système est un jour ouvert à des équipements tiers.

## 6 Tests et résultats

### 6.1 Protocole de test

Les essais ont été réalisés avec le matériel réel, dans l'ordre suivant :

1. compilation et téléversement du firmware sur l'ESP32 ;
2. ouverture du moniteur série à 115 200 bauds ;
3. démarrage du serveur avec `node server.js`, puis vérification du message « Serveur actif sur http ://0.0.0.0 :3000 » ;
4. ouverture du tableau de bord à l'adresse `http://localhost:3000` ;
5. observation pendant plusieurs minutes des trois interfaces : série, console serveur et navigateur ;
6. tests complémentaires de validation avec des requêtes HTTP volontairement invalides.

### 6.2 Journal série de l'ESP32

La figure 4 presente une première série d'envois. Chaque cycle affiche l'objet JSON envoyé, puis la réponse du serveur et le code HTTP. Les valeurs de température restent à 24,8 °C ; l'humidité oscille entre 64,7 % et 64,9 %. La quasi-totalité des requêtes reçoit un code 200, ce qui confirme la réception et le traitement correct des mesures.

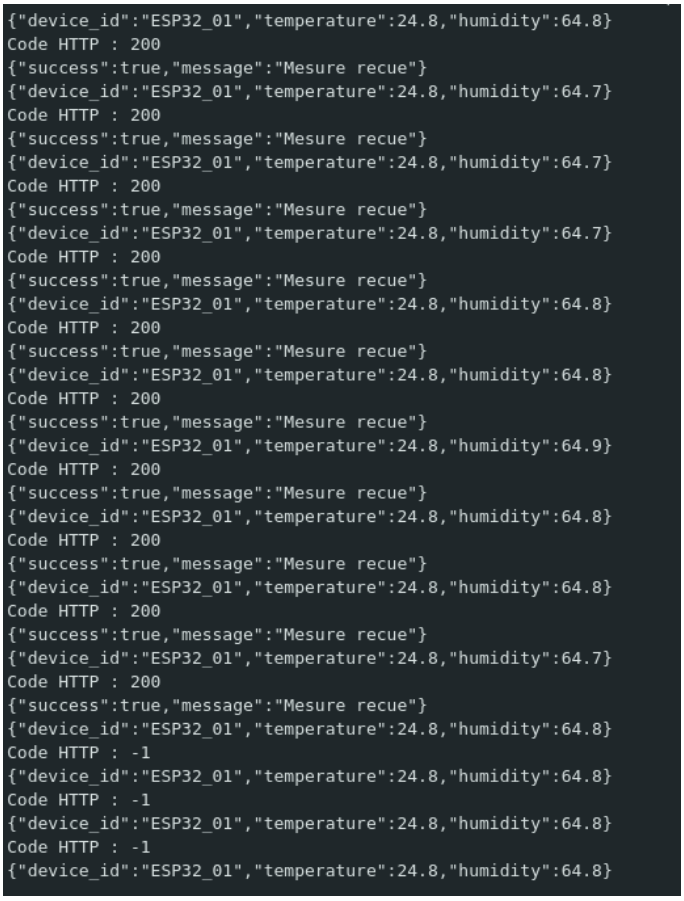



**Figure 4 –** Journal série de l'ESP32 : payloads JSON, réponses du serveur et codes HTTP.

En fin de série, plusieurs lignes affichent `Code HTTP : -1`. Ce code signifie que la requête n'a pas abouti : le serveur était momentanément injoignable, ou la connexion Wi-Fi a été interrompue. Le firmware se contente alors de journaliser l'échec et de poursuivre ; il ne réessaie pas. Cette observation justifie l'ajout d'une politique de reconnexion et d'un stockage local tampon dans une version ultérieure.

## 6.3 Journal du serveur

La figure 5 montre le journal du serveur pendant une seconde série. On y lit le message de démarrage, la connexion d'un tableau de bord, puis des objets `Mesure recue` contenant les champs normalisés et l'horodatage `receivedAt`. Les mesures sont reçues toutes les cinq secondes environ : 10 :56 :09, 10 :56 :14, 10 :56 :19, 10 :56 :24 et 10 :56 :29 UTC.

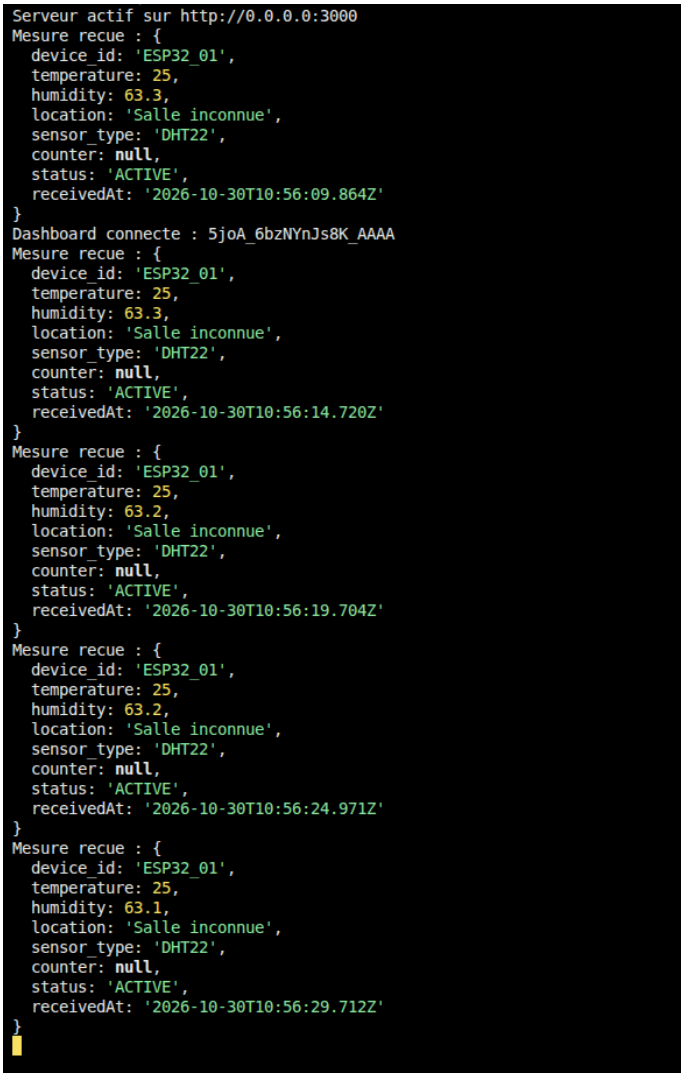



**Figure 5 –** Journal du serveur Node.js : démarrage, connexion du dashboard et mesures reçues.

Les valeurs de cette série sont stables : 25 °C et une humidité qui décroît lentement de 63,3 % à 63,1 %. La présence de `location: 'Salle inconnue'` et de `counter: null` confirme que le payload de test ne contenait que les trois champs minimaux : le serveur a appliqué ses valeurs par défaut, comme prévu par le code.

## 6.4 Tableau de bord en fonctionnement

La figure 3 montre l'interface au même moment. Le badge « Serveur connecté » est vert, la dernière réception affiche 11 :56 :14 (heure locale) et l'historique présente les mesures avec leur heure, leur température, leur humidité, leur salle et leur appareil. L'affichage est cohérent avec le journal serveur : 10 :56 :14 UTC correspond bien à 11 :56 :14 en UTC+1.

## 6.5 Synthèse des mesures

Le tableau 6 reprend les cinq mesures de la seconde série, avec l'écart entre deux réceptions successives. La période moyenne est de 4,96 s, proche de l'intervalle de 5 s programmé dans le firmware.

| # | Heure UTC | Heure locale | Température (°C) | Humidité (%) | Écart (s) |
|---|---|---|---|---|---|
| 1 | 10 :56 :09,864 | 11 :56 :09 | 25,0 | 63,3 | — |
| 2 | 10 :56 :14,720 | 11 :56 :14 | 25,0 | 63,3 | 4,86 |
| 3 | 10 :56 :19,704 | 11 :56 :19 | 25,0 | 63,2 | 4,98 |
| 4 | 10 :56 :24,971 | 11 :56 :24 | 25,0 | 63,2 | 5,27 |
| 5 | 10 :56 :29,712 | 11 :56 :29 | 25,0 | 63,1 | 4,74 |

**Table 6 –** Mesures de la seconde série et intervalles de réception.

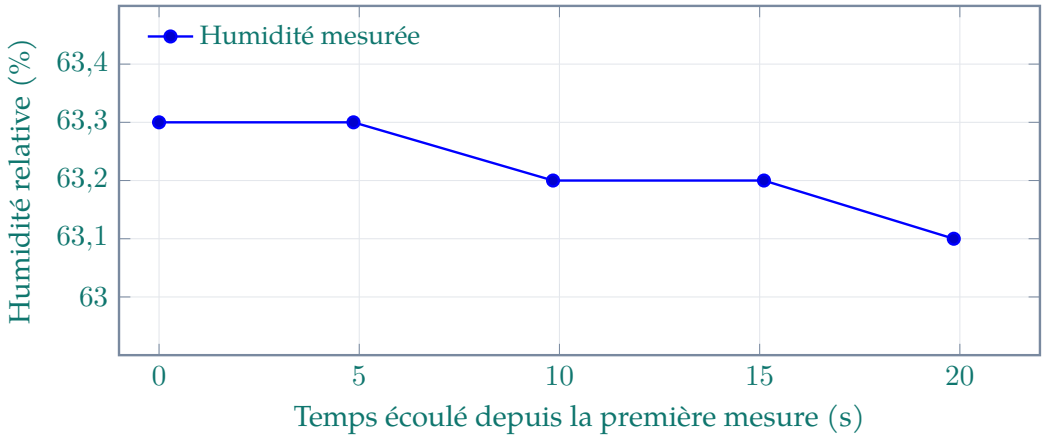



**Figure 6 –** Évolution de l'humidité pendant la seconde série ; la température reste constante à 25,0 °C.

La température est parfaitement stable sur cette série, tandis que l'humidité diminue de 0,2 point. Cette variation reste faible et cohérente avec un capteur de classe bas coût. Les écarts entre réceptions (4,74 à 5,27 s) s'expliquent par la latence réseau, le temps de traitement du serveur et l'arrondi des horodatages ; ils restent compatibles avec une acquisition toutes les cinq secondes.

## 6.6 Tests de validation

Des requêtes invalides ont été envoyées avec `curl` pour vérifier le comportement du serveur. Le tableau 7 résume les cas testés et la figure 8 donne les réponses obtenues.

| Cas | Donnée envoyée | Code |
|---|---|---|
| `device_id` absent | `{"temperature":24.8,"humidity":64.7}` | 400 |
| Température non numérique | `"temperature":"chaud"` | 400 |
| Humidité hors plage | `"humidity":120` | 400 |

**Table 7 –** Cas de validation testés.

```
HTTP 400
{"success":false,"message":"Donnees invalides",
 "errors":["device_id est obligatoire et doit etre une chaine de caracteres"]}

HTTP 400
{"success":false,"message":"Donnees invalides",
 "errors":["temperature doit etre un nombre"]}

HTTP 400
```

```
{"success":false,"message":"Donnees invalides",
 "errors":["humidity doit etre comprise entre 0 et 100 %"]}
```

**Code 8 –** Réponses du serveur aux requêtes invalides.

Les trois requêtes sont rejetées avec un code 400 et un message explicite. Aucune mesure invalide n'est ajoutée à l'historique, car le contrôle est effectué avant la construction de l'objet `measurement`. Ce comportement est conforme au cahier des charges.